

Aksha Support Chatbot — Phase 1 Evaluation Report

Date: 2026-08-24 · **Updated:** 2026-08-25 (MongoDB connectivity restored, full 30-query matrix re-run live — see Sections 3b and 3c)

Scope: All 6 implemented Phase 1 agents (Camera Operations, Alert Investigation, Live Monitoring, Insights & Analytics, Notification, Help & Product Guide) plus Multi-Language Support (formatter wrapper) and the React frontend.

Method: 30 scripted queries (5 per agent) run live against `aksha-chatbot-api` on `localhost:8010` via `POST /v1/chat/invoke` on 2026-08-24 (backend degraded), a 6-query spot-check on 2026-08-25 after the MongoDB fix, then the **full 30-query matrix re-run on 2026-08-25** against the fixed, genuinely reachable backend, plus manual live testing through the actual React UI on `localhost:5173`.

Raw data: `eval_queries.tsv` (inputs, shared across both runs), `eval_results.jsonl` (2026-08-24 outputs, degraded backend), `eval_results_live.jsonl` (2026-08-25 outputs, fixed backend).

1. Summary

Total queries run (2026-08-24 pass, degraded backend)	30 (5 per agent × 6 agents)
Correctly routed to the intended agent (2026-08-24)	30/30 , after one fix found during this pass (28/30 on the first run)
Agents that returned a live, grounded answer (2026-08-24)	help_guide only (5/5) — the other 5 agents were degraded, see below
Full 30-query matrix re-run on 2026-08-25, fixed backend	30/30 correctly routed, 30/30 resolved live — a completely clean pass
Bugs found and fixed on 2026-08-24	1 (router misclassification, live_monitoring vs. camera_operations)
Bugs found and fixed on 2026-08-25	2 (wrong <code>recentAlert</code> route on the deployed backend; 404/400 "not configured yet" misreported as an outage for notification & insights)
New bugs found in the full 30-query re-run	0 — all three earlier fixes held across the complete matrix; two minor answer-wording nuances noted (Section 3c), not functional bugs
Infrastructure blocker	<code>node_backend</code> /MongoDB connectivity — resolved 2026-08-25 , root cause and fix in Section 3b

Bottom line: the chatbot's routing and failure-handling logic is sound — every query reached the right agent (after the one fix the 2026-08-24 pass surfaced), and every agent that couldn't reach its backend failed *honestly*, with no invented data. As of 2026-08-25, the MongoDB connectivity blocker is fixed, and **the complete 30-query matrix has been re-run against the real backend with a clean 30/30 result** (Section 3c) — every agent, every query, resolved live with honest empty-state answers. This report now reflects a full data-accuracy pass, not just a spot-check.

2. Results by agent (2026-08-24 pass, degraded backend)

The tables below are the original 2026-08-24 run, made against the then-unreachable `node_backend` /MongoDB. Every query listed as "Degraded (TIMEOUT)" here now returns a real, honest empty-state answer instead — see Section 3c for the 2026-08-25 full re-run

against the fixed backend. Kept as-is for historical record and to show the routing/failure-handling behavior was already correct before the infrastructure fix.

2.1 Camera Operations

#	Query	Routed to	Status	Notes
1	How many cameras do we have configured?	camera_operations	Degraded (TIMEOUT)	Correct route; backend unreachable
2	Which cameras have email alerts enabled?	camera_operations	Degraded (TIMEOUT)	Correct route; backend unreachable
3	What camera groups exist and which cameras are in them?	camera_operations	Degraded (TIMEOUT)	Correct route; backend unreachable
4	What's the FPS setting on the loading dock camera?	camera_operations	Degraded (TIMEOUT)	Correct route; backend unreachable
5	List all cameras with person-detection enabled.	camera_operations	Degraded (TIMEOUT)	Correct route; backend unreachable

Routing: 5/5 correct. No degradation-handling issues — every failure produced the expected "3 retries then honest TIMEOUT message with a correlation id" pattern, never a hallucinated camera list.

2.2 Alert Investigation

#	Query	Routed to	Status	Notes
1	What alerts were triggered in the last 24 hours?	alert_investigation	Degraded (TIMEOUT)	Correct route; backend unreachable
2	Show me alerts for the main entrance camera.	alert_investigation	Degraded (TIMEOUT)	Correct route; backend unreachable
3	Were there any intrusion alerts yesterday?	alert_investigation	Degraded (TIMEOUT)	Correct route; backend unreachable
4	Give me the details of alert ID ALT-10293.	alert_investigation	Degraded (TIMEOUT)	Correct route; backend unreachable
5	Has camera 7 triggered any alerts this week?	alert_investigation	Degraded (TIMEOUT)	Correct route; backend unreachable

Routing: 5/5 correct.

2.3 Live Monitoring

#	Query	Routed to (1st run)	Routed to (after fix)	Notes
1	Show me the live cameras right now.	live_monitoring	live_monitoring	Correct
2	Which cameras are currently in spotlight view?	camera_operations ✗	live_monitoring ✓	Misrouted, then fixed — see Section 3

3	Is the warehouse camera online at this moment?	camera_operations ✗	live_monitoring ✓	Misrouted, then fixed — see Section 3
4	What's the current live status of all cameras?	live_monitoring	live_monitoring	Correct
5	Are any cameras currently down or offline?	live_monitoring	live_monitoring	Correct

Routing: 3/5 on the first run, 5/5 after the fix described in Section 3. All five, once correctly routed, returned the honest degraded response (backend unreachable).

2.4 Insights & Analytics

#	Query	Routed to	Status	Notes
1	Give me the insight report for the last 7 days.	insights_analytics	Degraded (TIMEOUT)	Correct route; backend unreachable
2	How many alerts happened this month compared to last month?	insights_analytics	Degraded (TIMEOUT)	Correct route; backend unreachable
3	What's the trend in false-positive alerts recently?	insights_analytics	Degraded (TIMEOUT)	Correct route; backend unreachable
4	Which camera generated the most alerts this week?	insights_analytics	Degraded (TIMEOUT)	Correct route; backend unreachable
5	Summarize alert activity for today.	insights_analytics	Degraded (TIMEOUT)	Correct route; backend unreachable

Routing: 5/5 correct.

2.5 Notification

#	Query	Routed to	Status	Notes
1	What are the email notification settings?	notification	Degraded (TIMEOUT)	Correct route (confirms the Section 9c fix held)
2	Who gets notified for camera group Warehouse-A?	notification	Degraded (TIMEOUT)	Correct route; backend unreachable
3	Do we get a notification every time an alert fires, or only for certain types?	notification	Degraded (TIMEOUT)	Correct route; backend unreachable
4	Was the notification for alert ALT-10293 delivered?	notification	Degraded (TIMEOUT)	Correct route; backend unreachable
5	List the notification recipients for the main gate group.	notification	Degraded (TIMEOUT)	Correct route; backend unreachable

Routing: 5/5 correct. Query 1 specifically re-validates the router fix made in Section 9c of the architecture doc (a prior pass had this query falling through to help_guide).

2.6 Help & Product Guide

This is the only agent whose backend (this repo's own docs, searched by `search_docs`) is actually reachable, so these are real, graded answers rather than degraded stubs.

#	Query	Routed to	Status	Answer quality
1	How do I search for an alert?	help_guide	Live, resolved	Good. Correctly cites <code>search_alerts / /api/alert</code> , <code>start_at / end_at</code> , <code>camera_ids</code> , <code>alert_types</code> — matches the real Node route contract.
2	How does multi-language support work?	help_guide	Live, resolved	Partially accurate. Correctly describes it as a read-only, no-new-API-surface feature, but repeats this doc's <i>target</i> -architecture "FAISS document index" language as if already built — the real implementation is a keyword/term-overlap MVP (Section 9b). Known, previously-flagged gap (Section 9c), reconfirmed here.
3	What does the spotlight view feature do?	help_guide	Live, resolved	Correct behavior, content gap. Answered "I'm not able to confirm that based on the available documentation" rather than guessing — exactly right when the docs corpus has no page on this feature. The gap is in documentation coverage, not agent logic.
4	How do camera groups relate to notification settings?	help_guide	Live, resolved	Good. Correctly cites <code>/api/camgroup</code> and <code>/api/notification</code> as separate route families, explains the group-ID linking relationship, and is honest that the docs don't specify more detail.
5	What can this chatbot help me with?	help_guide	Live, resolved	Good. Reasonable, on-topic capability summary grounded in the docs corpus.

Routing: 5/5 correct. Answer quality: 4/5 fully correct, 1/5 correct-but-stale on a known, already-documented gap.

3. Bug found and fixed during this evaluation

Router misclassification: `live_monitoring` vs. `camera_operations`.

"Which cameras are currently in spotlight view?" and "Is the warehouse camera online at this moment?" both misrouted to `camera_operations`. Root cause: `app/router.py`'s disambiguation rules distinguished `camera_operations` from notification and from `help_guide`, but had no rule at all separating it from `live_monitoring` — so a present-moment-status question that happens to name a camera fell through to the configuration agent.

Fix: added an explicit rule to the router's system prompt — phrasing like "currently", "right now", "at this moment", or "spotlight view" routes to `live_monitoring` (present-moment status), while `camera_operations` stays scoped to static configuration (settings, groups, FPS, detection features), even when a specific camera is named.

Verification: both queries re-tested after the fix and now route correctly; the original `camera_operations` regression case ("Which cameras have email alerts enabled?") was re-tested alongside and still routes correctly — no regression introduced.

This is the third router-disambiguation bug found through live testing across Phase 1's build and evaluation (the first two are documented in Section 9b/9c of the architecture doc), all following the same pattern: the router prompt is only as good as its explicit disambiguation rules, and each new agent added a new pairwise ambiguity that needed its own rule rather than being inferred by the model.

3b. MongoDB Connectivity Fixed, All 6 Agents Verified Live (2026-08-25)

The infrastructure blocker behind Section 1's "25/25 degraded" result is now fixed, and every agent has been re-verified against a genuinely reachable backend for the first time in this project.

Root cause: not a DNS server bug as long assumed. `docker inspect mongodb` showed `NetworkSettings.Networks: {}` — the `mongodb` container had no network endpoint attached at all, despite its config correctly naming `aksha-net`. `node_backend` couldn't resolve `mongodb` because the container genuinely wasn't joined to any network. A plain `docker restart mongodb` did not fix it; `docker network connect aksha-net mongodb` did, and restarting `node_backend` afterward produced a clean `Connected to MongoDB` in its logs.

Two new bugs found once real connectivity existed:

- 1. **Wrong route.** `get_recent_alerts` called `/api/recentAlert/:hours`, which 404s on the actually-deployed backend image (`dockerhubalgo/aksha_refactor_backend` — a different, newer codebase than the `AkshaV2-UIUX/backend` source this project's tool contracts were built against). Black-box probing found the real contract needs a second path segment: `/api/recentAlert/:hours/all`. Fixed in `app/tools_alerts.py`.
- 2. **Misleading error wording.** `/api/email_notification` and `/api/insightReport` both return 4xx for the legitimate case of "nothing configured/no data yet" in this unseeded dev database, which the chatbot's uniform 4xx-handling was reporting as "service unavailable, try again" — true-sounding but wrong, since retrying changes nothing. Fixed in `app/tools_notification.py` and `app/tools_insights.py` to recognize these specific empty-state responses and answer honestly instead ("not configured yet" / "no data for that range").

Live re-verification, one query per agent, real (empty) database:

Agent	Query	Result
camera_operations	How many cameras do we have configured?	<code>resolved / live</code> — "You have 0 cameras configured."
alert_investigation	What alerts were triggered in the last 24 hours?	<code>resolved / live</code> — "No alerts were triggered in the last 24 hours."
live_monitoring	Show me the live cameras right now.	<code>resolved / live</code> — "There are no live cameras available at the moment."
insights_analytics	Give me the insight report for the last 7 days.	<code>resolved / live</code> — "the system returned no camera data"
notification	What are the email notification settings?	<code>resolved / live</code> — "Email notifications are not currently configured."
help_guide	How do I search for an alert?	<code>resolved / live</code> — correctly cites <code>search_alerts / /api/alert</code> (unchanged from 2026-08-24)

All six now return `status: resolved, freshness.kind: live`. These are honest empty-state answers, not fabricated data — correct behavior for a dev database with nothing seeded, and exactly what should happen.

Also re-verified through the actual UI (not just the API): the same live query flow was exercised through the real composer in Live mode, confirming the agent tile, "Live" freshness pill (accent-100 background, as

specified), and streamed answer all render correctly for a genuinely live response — not just the scripted demo transcript.

Full detail, including the diagnostic steps and the exact `docker` commands used: Section 9e of `CHATBOT_SYSTEM_ARCHITECTURE.md`.

3c. Full 30-Query Matrix Re-Run, Live Backend (2026-08-25)

With connectivity fixed, the exact same 30 queries from Section 2 (same `eval_queries.tsv`) were re-run end-to-end against the real backend. Full raw output: `eval_results_live.jsonl`.

Result: 30/30 correctly routed, 30/30 status: resolved with freshness.kind: live. Every query that was previously a "correct route, honest TIMEOUT" degraded response in Section 2 now returns a real, honest empty-state answer instead — none fabricated data, and the router disambiguation fix from Section 3 held with zero regressions across all 30 queries (not just the 5 Live Monitoring queries it was originally tested against).

Representative examples, one per agent (full 30 in the raw JSONL):

Agent	Query	Live answer
camera_operations	How many cameras do we have configured?	"We have zero cameras configured at the moment."
alert_investigation	Give me the details of alert ID ALT-10293.	"I'm unable to locate any details for alert ALT-10293 at this time."
live_monitoring	Are any cameras currently down or offline?	"I can't confirm the status of any cameras right now because the live-camera list returned no data."
insights_analytics	Summarize alert activity for today.	"No alert activity is recorded for today; the insight report returned no camera entries."
notification	List the notification recipients for the main gate group.	"The notification configuration for the main gate group is not set up, so there are currently no recipients defined."
help_guide	What can this chatbot help me with?	On-topic capability summary, unchanged from the 2026-08-24 pass (this agent doesn't depend on <code>node_backend</code>).

Two answer-wording nuances found, neither a functional bug:

1. **"Warehouse-A" framed as "disabled" rather than "doesn't exist."** For "Who gets notified for camera group Warehouse-A?", the answer was "No users or groups are configured... the notification setting is disabled." But this dev database has zero camera groups defined at all — "Warehouse-A" isn't a real group that happens to have notifications off, it doesn't exist. The tool's normalized empty-state response (Section 3b's fix) doesn't currently distinguish "this specific group has no config" from "no group with this name exists," and the formatter's prose picked the more specific-sounding (but less accurate) framing. Worth a follow-up: have `get_group_notification` or the agent check the group against `get_camera_groups` first, or soften the wording to "no group found or no config."

2. A **"try again" suggestion for a range that can't change**. For "How many alerts happened this month compared to last month?", the answer included "Please try retrieving the insight report again" — but the underlying issue is that no report data exists for this date range at all (an empty dev database), not a transient failure, so retrying the same range won't help. This is an LLM phrasing choice in the formatter, not a code defect, but it's the same class of "implies retry helps when it doesn't" issue Section 3b fixed for the raw tool-error path — worth a small system-prompt note telling agents not to suggest retrying for a confirmed-empty result.

Neither issue affects routing, tool correctness, or whether the answer is honest about having no data — both are precision-of-wording notes for a future polish pass.

What this confirms that Section 3b's 6-query spot-check could not: every one of the original 30 test queries — not just one representative per agent — now resolves correctly against live infrastructure. This closes the main gap flagged in the previous version of this report.

4. UI verification (manual, through the actual React frontend)

Beyond the scripted API evaluation, the chat was exercised through the real UI at `localhost:5173`:

- **DEMO/LIVE toggle** (added during this session): confirmed Demo mode shows the untouched scripted design-spec transcript, Live mode starts from an empty state with suggestion chips, and sending a message from either mode correctly hops to Live and calls the real backend — with no cross-contamination between the two message lists.
- **Live streaming end-to-end**: sent "How do I search for an alert?" through the real composer; the UI correctly showed the "routed to help_guide" status, streamed the answer token-by-token via SSE, and rendered the final `HELP_GUIDE` agent badge with a "LIVE" freshness indicator — matching the API-level result for the same query.
- **Degraded-state rendering**: a `live_monitoring` query sent through the UI while the backend is down is expected to render the app's "degraded" message variant (retry button, correlation id) rather than the scripted demo's mock degraded example — consistent with the API-level TIMEOUT behavior confirmed above.

5. What this evaluation still has not verified

1. **Behavior against a populated database**. Every live answer confirmed in Sections 3b and 3c is an honest *empty*-state response — this dev database has zero cameras, alerts, or notification configs seeded. Whether the formatter's prose stays accurate with real volume (many cameras, many alerts, actual recipients, genuine ambiguity between similarly-named cameras) is unverified. This is the one remaining real gap in data-accuracy coverage.
2. **The two answer-wording nuances from Section 3c** (a nonexistent group framed as "disabled" rather than "not found"; a "try again" suggestion for a permanently empty date range) are noted but not yet fixed.
3. **Multi-Language Support** was not included in the 30-query set (it's a formatter wrapper, not a routable agent as of the Section 9c fix) — its translation behavior was already verified separately with a live Hindi test in Section 9b and is not re-tested here.

6. Recommendation

Phase 1's chatbot logic (routing, tool-calling, graceful degradation, streaming, translation) is ready to sign off on. As of 2026-08-25, it has been confirmed working end-to-end against real backend connectivity across the complete 30-

query test matrix — not just a spot-check, and not just its failure path — with zero new bugs and only two minor wording nuances found. Remaining open items: a populated-database check (Section 5), the two wording-nuance fixes (Section 3c), and a documentation-hygiene pass so `search_docs` doesn't present target architecture as shipped fact (Sections 9c, 10 of the architecture doc). None of these are functional defects — they're verification depth and polish work.